

附录 Tiger 语言参考手册

Tiger 语言是一个小规模的语言，它有嵌套函数、采用隐含指针的记录值、数组、整型和字符串变量，以及几种简单的结构化的控制结构。

A.1 词法名词

标识符：标识符是以字母开头，由字母、数字和下划线组成的序列（区分大小写字母）。在本附录中，符号 *id* 代表一个标识符。

注释：注释可以出现在任意两个单词之间。注释以 */** 开始，以 **/* 结束，并且可以嵌套。

A.2 声明

声明序列是由一系列的类型、值和函数声明组成的序列；各个声明之间没有用来分隔或终止一个声明的标点符号。

decs → {*dec*}

dec → *tydec*
→ *vardec*
→ *fundec*

本节所用的语法记号中， ϵ 代表空字符串， $\{x\}$ 代表可能为空的序列 x 。

数据类型

Tiger 中类型和类型声明的语法是

tydec → **type** *type-id* = *ty*

ty → *type-id*
→ { *tyfields* } (这里的花括号代表花括号自身)
→ **array of** *type-id*

tyfields → ϵ
→ *id* : *type-id* {, *id* : *type-id*}

- 内建类型：有两个预先定义的命名类型 *int* 和 *string*。可以通过类型声明定义或重新定义（包括那些预定义的）其他的命名类型。
- 记录：记录类型是由花括号中列出的它们的各个域来定义的，每一个域用 *fieldname* : *type-id* 来描述，其中 *type-id* 是一个由类型声明定义的标识符。
- 数组：任何命名类型构成的数组可以通过 **array of** *type-id* 来创建。数组的长度不作為这个类型的一部分被指定；这个类型的每一个数组都可能有不同的长度，并且长度是在程序

运行中创建数组时确定的。

- **记录各不相同**：每一个记录或数组类型的声明创建一个新的类型，并且这个新类型与其他记录或数组类型不兼容（即使所有的域都相同）。
- **相互递归的类型**：一组类型可以递归或相互递归。相互递归的类型是通过一系列连续的、其间没有介入值或函数声明的类型声明来指明的。每一个递归环必须经过一个记录或数组类型。

因此，整数表的类型是：

```
type intlist = {hd: int, tl: intlist}
type tree = {key: int, children: treelist}
type treelist = {hd: tree, tl: treelist}
```

但是，下面的声明序列是非法的：

```
type b = c
type c = b
```

- **域名的可重用性**：不同的记录类型可以使用相同的域名（如上例中 intlist 和 treelist 的域 hd）。

519

A. 2.1 变量

```
vardec → var id := exp
       → var id : type-id := exp
```

在上面的变量声明中，短形式的声明给出了变量名，其后跟随一个表示该变量初值的表达式。在这种形式中，变量的类型取决于这个表达式的类型。

在长形式的变量声明中，同时还给出了变量的类型。初值表达式必须具有相同的类型。

如果初值表达式是 **nil**，则必须使用长形式的声明。

每一个变量声明创建一个新的变量，它的生命期同其声明的作用域一样长。

A. 2.2 函数

```
fundef → function id ( tyfields ) = exp
       → function id ( tyfields ) : type-id = exp
```

上面的第一行是一个过程声明；第二行是一个函数声明。过程没有返回值；但函数返回结果值，并且结果值的类型在冒号之后指明。*exp* 是过程体或函数体，*tyfields* 指明了参数的类型和名字。所有参数都是传值参数。

函数可以递归。相互递归的函数和过程通过一系列连续的函数声明来指明（之间没有插入类型或变量声明）：

```
function treeLeaves(t : tree) : int =
    if t=nil then 1
    else treelistLeaves(t.children)
function treelistLeaves(L : treelist) : int =
    if L=nil then 0
    else treeLeaves(L.hd) + treelistLeaves(L.tl)
```

A. 2.3 作用域规则

- **局部变量**：在表达式 `let ... vardec ... in exp end` 中声明的变量的作用域从 `vardec` 之后开始直到 `end` 结束。
- **参数**：在 `function id( ... id1:id2...) = exp` 中，参数 `id1` 的作用域是整个函数体 `exp`。
- **嵌套作用域**：变量或参数的作用域也包括那个作用域中每一个函数定义的函数体。也就是说，Tiger 同 Pascal 和 Algol 一样，允许访问外层作用域中的变量。
- **类型**：在表达式 `let ... tydec ... in exps end` 中，类型标识符的作用域从定义它的类型声明的连续序列开始，一直延续到 `end`。这包括此作用域内所有函数的函数头和函数体。
- **函数**：在表达式 `let ... fundecs ... in exps end` 中，函数标识符的作用域从定义它的函数声明的连续序列开始，一直延续到 `end`。这包括此作用域内所有函数的函数头和函数体。
- **名字空间**：有两类不同的名字空间：一种是类型的名字空间，另一种是函数和变量的名字空间。类型 `a` 可以与变量 `a` 或函数 `a` 同时处在一个作用域中，但是，同名的变量和函数不能同时处于一个作用域中（其中一个将隐藏另一个）。
- **局部重声明**：变量或函数的声明可以被较小作用域中同名的（变量或函数的）重复声明所隐藏。例如，以参数 5 调用下面这个函数将输出 “6 7 6 8 6”：

```
function f(v: int) =
let var v := 6
in print(v);
  let var v := 7 in print (v) end;
  print(v);
  let var v := 8 in print (v) end;
  print(v)
end
```

函数可隐藏同名变量，反之亦然。类似地，类型声明可以被其作用域内具有较小作用域的同名的重复声明所隐藏。但是，在相互递归的函数序列中，不能有同名的函数；并且在相互递归的类型序列中，不能有同名的类型。

A. 3 变量和表达式

A. 3.1 左值

左值是可以从其中取出值或对其赋值的一个位置。变量、过程参数、记录域和数组元素都是左值。

```
lvalue → id
       → lvalue . id
       → lvalue [ exp ]
```

- **变量**：形如 `id` 的标识符引用一个根据作用域规则可访问的变量或参数。
- **记录域**：点号表示法允许选择一个记录值相应的命名域。
- **数组下标**：方括号表示法允许选择与编号对应的数组元素。数组以从 0 开始的连续整数（最大值为数组大小减 1）作为索引。

A.3.2 表达式

- **左值**：当用于表达式时，其值是它对应的位置中的内容。
- **无值表达式**：有一些表达式不产生结果，包括过程调用、赋值、*if-then*、*while*、*break*，有时还有 *if-then-else*。因此，尽管表达式 $(a := b) + c$ 在词法上是正确的，但它却通不过类型检查。
- **nil**：表达式 **nil**（保留字）表示一个属于所有记录类型的值 **nil**。如果记录变量 *v* 的值为 **nil**，则从 *v* 选择一个域会检测出一个运行时错误。**nil** 必须用在可以确定其类型的上下文中，也就是：

var a : my_record := nil	OK
a := nil	OK
if a <> nil then ...	OK
if nil <> a then ...	OK
if a = nil then ...	OK
function f(p: my_record) = ... f(nil)	OK
var a := nil	非法
if nil = nil then ...	非法

- **序列**：括在括号内用分号隔开的两个或两个以上表达式组成的序列 (*exp*; *exp*; ... *exp*)，按排列顺序计算它的所有表达式。此序列的结果是最后一个表达式产生的结果（如果有结果的话）。
- **无值**：一个开括号其后跟随一个闭括号（这两个括号是两个独立的单词）是一个不产生值的表达式。类似地，在 **in** 和 **end** 之间没有内容的 **let** 表达式也不产生值。
- **整型字面量**：由十进制数字组成的一个序列是一个代表对应整数值的整型常数。
- **字符串字面量**：字符串是一个序列，它由括在双引号之间的 0 或多个可打印字符、空白符或转义序列组成。每一个转义序列由转义字符 \ 引入，代表一个字符序列。Tiger 允许有如下的转义序列（\ 的所有其他用法都是非法的）。

522

\n	系统中表示换行的字符。
\t	制表符 Tab。
\^c	控制字符 c，适用于任何适当的字符 c。
\ddd	具有 ASCII 码 ddd（3 个十进制数字）的单个字符。
\"	双引号字符 (")。
\\	反斜线字符 (\)。
\f__f\	此序列将被忽略。其中 f__f 代表一个或多个以上的格式化字符（非可打印字符的子集，至少应包含空白符、制表符、换行符、换页符）组成的序列。这使我们可以在一行的末尾和下一行的开始各写一个 \，从而写出长度超过一行的长字符串。

- **负值**：整型值表达式之前可以带有一个负号。
- **函数调用**：函数调用 *id()* 或 *id(exp{,exp})* 表示从左至右计算实参表，并用计算出的实参值来调用函数 *id*。这些实参与该函数定义的对应该形参相结合，函数体按照传统的静态作用域规则计算出一个结果。如果 *id* 实际表示的是一个过程（即无返回值的函数），则其函数体不能产生结果值，并且调用此函数也不会有返回结果。

- **算术操作**: 形如 $exp\ op\ exp$ 的表达式, 其中 op 是 $+$ 、 $-$ 、 $*$ 、 $/$, 要求整型操作数, 并且产生一个整型结果。
- **比较**: 形如 $exp\ op\ exp$ 的表达式, 其中 op 是 $=$ 、 $<>$ 、 $>$ 、 $<$ 、 $>=$ 、 $<=$, 比较它的两个操作数的相等或不等性, 并在比较结果为真的情况下产生整数 1, 在为假的情况下产生整数 0。所有这些操作符都可以应用于整型操作数。相等或不等操作符也可应用于相同类型的两个记录或数组操作数, 但比较的是其“引用”或“指针”的相等性 (它们测试的是两个记录是否是相同的实例, 而不是这两个记录是否具有相同的内容)。
- **字符串比较**: 比较操作符也可应用于字符串。如果两个字符串的内容相同, 则这两个字符串相等; 没有办法识别部分字符相同的两个字符串。不等性是按照词典序来比较的。
- **布尔操作**: 形如 $exp\ op\ exp$ 的表达式, 其中 op 是 $\&$ 或 $|$, 表示按捷径方式计算布尔交和并: 当它们可以由左操作数确定出其结果时, 将不再计算其右操作数。任何非 0 的整数值都看成真值, 整数 0 是假值。
- **操作符的优先级**: 一元负 (取负) 具有最高的优先级, 操作符 $*$ 、 $/$ 具有次高优先级, 其次是 $+$ 、 $-$, 之后是 $=$ 、 $<>$ 、 $>$ 、 $<$ 、 $>=$ 、 $<=$, 再之后是 $\&$, 最后是 $|$ 。
- **操作符的结合性**: 操作符 $*$ 、 $/$ 、 $+$ 、 $-$ 都是左结合的。比较操作符不能结合, 因此, 尽管 $a = (b = c)$ 是合法的, $a = b = c$ 不是合法的表达式。
- **记录创建**: 表达式 $type-id\{id = exp\}$, $id = exp\}$ 或 (对于一个空记录类型) $type-id\{\}$ 创建一个类型为 $type-id$ 的新的记录实例。该记录表达式的域名和类型必须按给定的顺序与命名类型的域名和类型相匹配。这里的花括号 $\{\}$ 就是花括号自身。
- **数组创建**: 表达式 $type-id[exp_1]\ of\ exp_2$ 按顺序计算 exp_1 和 exp_2 , 分别计算出其元素的个数 n 和初始值 v 。类型 $type-id$ 必须声明为数组类型。该表达式的结果是一个类型为 $type-id$, 索引范围从 0 到 $n-1$ 的新数组, 此数组的每一个元素的初值都为 v 。
- **数组和记录赋值**: 当一个数组或记录变量 a 被赋予了一个值 b 时, a 引用的是与 b 相同的数组或记录。之后对 a 的元素的更新将影响 b , 反之亦然, 直到 a 被重新赋值。数组和记录参数传递的是地址, 而不是值。
- **生存期**: 记录和数组具有无限的生存期: 每一个记录或数组值是永久存在的, 即使控制已退出了声明它们的作用域也如此。
- **赋值**: 赋值语句 $lvalue := exp$ 先计算 $lvalue$, 接着计算 exp , 然后设置 $lvalue$ 的内容为表达式 exp 的结果。在句法上, $:=$ 的优先级低于布尔操作符 $\&$ 和 $|$ 。赋值表达式不产生值, 所以 $(a := b) + c$ 是非法的。
- **if-then-else**: if 表达式 $if\ exp_1\ then\ exp_2\ else\ exp_3$ 计算整型表达式 exp_1 。如果结果不为 0, 则产生计算表达式 exp_2 的结果; 否则产生 exp_3 的结果。表达式 exp_2 和 exp_3 必须具有相同的类型, 此类型也是整个 if 表达式的类型 (或者, 两个表达式都必须是无值的)。
- **if-then**: if 表达式 $if\ exp_1\ then\ exp_2$ 计算整型表达式 exp_1 。如果结果不为 0, 则计算 exp_2 (它必须不产生值)。整个 if 表达式是无值的。
- **while**: 表达式 $while\ exp_1\ do\ exp_2$ 计算整型表达式 exp_1 。如果结果不为 0, 则计算 exp_2 (它必须是无值的), 然后重新计算整个 $while$ 表达式。
- **for**: 表达式 $for\ id := exp_1\ to\ exp_2\ do\ exp_3$ 对取值范围在 exp_1 和 exp_2 之间的 id 的每一个整数值重复计算 exp_3 。变量 id 是一个由该 for 语句隐含声明的新变量, 它的作用域仅在 exp_3 中, 并且在 exp_3 内不可以对它赋值。循环体 exp_3 必须是无值的。循环的上界 exp_1 和

下界 exp_2 只在进入循环体之前计算一次。如果上界小于下界，则不会执行循环体。

- **break:** **break** 表达式终止直接包含它的那个 **while** 表达式或 **for** 表达式的计算。即使 p 嵌套在 q 之内，过程 p 之内的 **break** 也不能终止过程 q 中的循环。**break** 位于 **while** 或 **for** 之外是非法的。
- **let:** 表达式 **let decs in expseq end** 计算声明 *decs*，绑定类型、变量和过程使它们的作用域为整个 *expseq*。*expseq* 是 0 个或多个用分号分隔的表达式所形成的序列，此序列中最后一个表达式的结果（若有的话）将作为整个 **let** 表达式的结果。
- **圆括号:** 和大多数程序设计语言一样，括住任何表达式的圆括号都强制它们在句法上组成一组。

A.3.3 程序

Tiger 程序没有参数：程序就是一个表达式 *exp*。

A.4 标准库

Tiger 有下面几个预先定义的函数。

```
function print(s : string)
```

输出 *s* 至标准输出。

```
function flush()
```

排空标准输出缓冲区。

```
function getchar() : string
```

从标准输入读一个字符；遇到文件尾则返回空字符串。

```
function ord(s: string) : int
```

给出 *s* 中第一个字符的 ASCII 值；如果 *s* 是空字符串，则返回 -1。

```
function chr(i: int) : string
```

ASCII 值为 *i* 的单字符字符串；若 *i* 超出了 ASCII 字符集的范围，程序将停止。

```
function size(s: string) : int
```

s 中字符的个数。

```
function substring(s:string, first:int, n:int) : string
```

字符串 *s* 中从第 *first* 个字符开始、长度为 *n* 的子字符串，字符从 0 开始编号。

```
function concat (s1: string, s2: string) : string
```

s1 和 *s2* 的串联得到的字符串。

```
function not(i : int) : int
```

返回 (*i* = 0)。

```
function exit(i: int)
```

以状态码 i 终止程序的执行。

A.5 Tiger 程序示例

本节给出了两个已完成的 Tiger 程序；程序 6-2 只是一个 Tiger 程序的一部分（一个函数）。

A.5.1 QUEENS. TIG

```
/* A program to solve the 8-queens problem */
let
  var N := 8

  type intArray = array of int

  var row := intArray [ N ] of 0
  var col := intArray [ N ] of 0
  var diag1 := intArray [N+N-1] of 0
  var diag2 := intArray [N+N-1] of 0

  function printboard() =
    (for i := 0 to N-1
     do (for j := 0 to N-1
        do print(if col[i]=j then " O" else " .");
        print("\n"));
     print("\n"))

  function try(c:int) =
    if c=N
    then printboard()
    else for r := 0 to N-1
        do if row[r]=0 & diag1[r+c]=0 & diag2[r+7-c]=0
            then (row[r]:=1; diag1[r+c]:=1; diag2[r+7-c]:=1;
                col[c]:=r;
                try(c+1);
                row[r]:=0; diag1[r+c]:=0; diag2[r+7-c]:=0)

  in try(0)
end
```

这个程序输出了所有满足如下要求的棋盘布局：在国际象棋棋盘上放置 8 个皇后，使之满足同一行、同一列和同一对角线上不会有两个皇后。它说明了数组和递归的用法。假设我们已成功地在第 $0 \sim c-1$ 列放置了两个皇后，则当第 r 行放置了皇后时， $row[r]$ 将是 1；当第 d 条左下角至右上角的对角线放置了皇后时， $diag1[d]$ 将是 1；当第 d 条左上角至右下角的对角线放置了皇后时， $diag2[d]$ 将是 1。接下来， $try(c)$ 尝试在第 $c \sim N-1$ 行放置皇后。

A.5.2 MERGE. TIG

```
let  type any = {any : int}
     var buffer := getchar()

function readint(any: any) : int =
  let var i := 0
     function isdigit(s : string) : int =
```



```

    ord(buffer)>=ord("0") & ord(buffer)<=ord("9")
in while buffer=" " | buffer="\n" do buffer := getchar()
    any.any := isdigit(buffer);
    while isdigit(buffer)
        do (i := i*10+ord(buffer)-ord("0");
            buffer := getchar());
    i
end

type list = {first: int, rest: list}

function readlist() : list =
    let var any := any{any=0}
        var i := readint(any)
    in if any.any
        then list{first=i,rest=readlist()}
        else (buffer := getchar(); nil)
    end

function merge(a: list, b: list) : list =
    if a=nil then b
    else if b=nil then a
    else if a.first < b.first
        then list{first=a.first,rest=merge(a.rest,b)}
        else list{first=b.first,rest=merge(a,b.rest)}

function printint(i: int) =
    let function f(i:int) = if i>0
        then (f(i/10); print(chr(i-i/10*10+ord("0"))))
    in if i<0 then (print("-"); f(-i))
        else if i>0 then f(i)
        else print("0")
    end

function printlist(l: list) =
    if l=nil then print("\n")
    else (printint(l.first); print(" "); printlist(l.rest))

/* BODY OF MAIN PROGRAM */
in printlist(merge(readlist(), readlist()))
end

```

这个程序从标准输入读入两列整数；每列中的数应按递增顺序排列，数之间用空白或换行符分隔；每一列数应当用分号来终止。

程序的输出是这两列数的合并：即一列按递增顺序排列的数。

记录 any 在 Tiger 中用来模拟传地址调用。尽管 readint 不能更改它的实参（以指明输入中是否还余留有要输入的数），但可以更改它的实参的一个域。

赋值 any := any{any=0} 说明了一个名字可以表示变量、类型和域，具体表示什么取决于它的上下文。